```
// Rewrite: one CTE per branch (AGG[COUNT(*) GROUP BY fk]);
// root → AGG[SUM(b_1.n × ··· × b_N.n)].
LogicalOperatorPtr JoinDecomposeToPreaggregatedCount::Apply(
 LogicalOperatorPtr op) const {
 /* collect scans/preds, split per-alias, find anchor_pk */
 auto cte_op = std::make_unique<CTEOperator>();
 for (const auto &br : branches)
  cte_op->AddCTE(br.cte_name, buildCountCTE(
   br.scan.table_name, br.scan.alias,
   br.fk_col, scan_preds[br.scan.alias]));
 LogicalOperatorPtr tree =
  std::make_unique<ScanOperator>(anchor_table, anchor);
 for (const auto &br : branches) {
  auto j = std::make_unique<JoinOperator>(JoinType::INNER,
   std::vector<std::string>{anchor+"."+anchor_pk
                    +" = "+br.scan.alias+"."+br.fk_col});
  j->AddChild(std::move(tree));
  j->AddChild(std::make_unique<ScanOperator>
      (br.cte_name, br.scan.alias));
  tree = std::move(j); }
 std::string sum_expr = "sum(";
 for (std::size_t i = 0; i < branches.size(); ++i) {
  if (i > 0) sum_expr += " * ";
  sum_expr += branches[i].scan.alias + ".n"; }
 sum_expr += ")";
 auto final_agg = std::make_unique<AggregateOperator>(
  std::vector<std::string>{}, std::vector<std::string>{sum_expr});
 final_agg->AddChild(std::move(tree));
 cte_op->SetFinalQuery(std::move(final_agg));
 return cte_op; }
```